不绘制视野外的元素？

> 在浏览器中，绘制消耗的性能是大于纯计算的。
>
> - 计算：主要涉及到 `JavaScript` 的执行，这通常在浏览器的 `JavaScript` 引擎（如 V8引擎）中进行。这些引擎都经过了高度优化，可以快速执行代码。而且，许多计算任务可以并行执行，可以进一步提高性能。
> - 绘制：对于绘制而言，它涉及到浏览器的渲染引擎，它需要将 `HTML`、 `CSS` 和 `JavaScript` 转化为用户可以看到和交互的图形界面。这个过程包括布局（计算元素的位置和大小）、绘制（将元素绘制到屏幕上）和合成（将多个层合并到一起，形成最终的图像）。这些步骤都需要大量的计算和内存，因此通常比纯计算更消耗性能。
>
> 所以如果我们能通过简单的**纯计算来减少不必要的页面重绘**，那对于页面的性能提升来说未尝不是一件值得尝试的事情。

思考：我们能否在每次绘制前通过计算判断元素是否在画布视野外来取消元素的绘制，来达到减少图像绘制时的像素计算？

这个问题的提出其实建立在图案超出画布的视野后还是会绘制的前提上，所以要找到这个问题的答案，我们首先需要先找到另一个问题的答案。

画布视野外的元素是否会参与像素计算？

① 第一次测试：测试元素超出视野是否会有更快的绘制速度

https://codepen.io/huanjinliu/embed/zYMPJqP?editors=1111

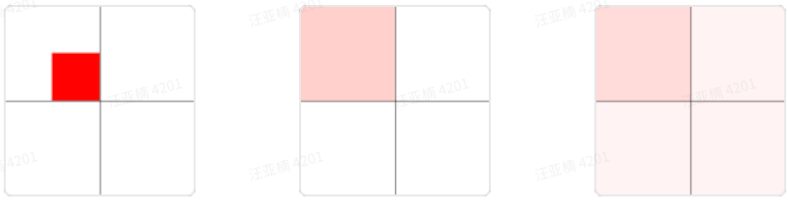
从测试结果上看，当元素超出视野后，绘制的速度确实会快上许多，但这仍不能说明超出像素未参与计算，虽然整体速度快了，但却仍需要较长的绘制时间，比起单纯的计算界限超出所耗费的时间要多得多。

② 第二次测试：运用“浮点数”抗锯齿的特性：

接下来我们来看看下面的例子：

```
const canvas = document.createElement('canvas');
canvas.width = 2;
canvas.height = 2;
const ctx = canvas.getContext('2d');

ctx.fillStyle = 'rgb(255, 0, 0)';
ctx.fillRect(0.5, 0.5, 0.5, 0.5);
```



①理想的效果　②铺满效果　③平滑后效果

上面的例子如果我们将绘制的起点从 `(0.5, 0.5)` 改成 `(-0.5, -0.5)` 如果仍能看到第一个格子有填充色，这时我们可以认为超出界限的像素是会参与像素计算的。

👇这个例子默认还是（0.5, 0.5）

https://codepen.io/huanjinliu/embed/bGQYmEr

从测试结果上看，我们可以认为 `canvas` 对于超出边界的图像是不做像素计算的。

后经查阅，画布绘制时，绘图引擎会自动剪裁超出部分，因为这可以减少不必要的计算和绘制，从而提高绘制性能。

示例一中仍然速度比较慢是因为 `drawImage` 底层逻辑在判断是否裁剪前还做了其他如：参数检查、图像解码的操作，所以导致绘制超边界的图像时仍需要耗费一定时间。所以即使浏览器本身自带优化，但 `canvas` 的API本身就像个黑盒子，很多额外操作我们是不清楚的，手动通过计算判断元素是否在画布视野外来取消元素的绘制这一步在某些情况下仍然是有必要的。